

Loss based methods for OCGIN node+graph level

12. April 2026

1 Briefing

This is a document used for dropping experimental results from my bachlor thesis in a somewhat structured form but still informal.. form. here you will find mathematical description of methods, ran experiments and their results

2 Constrains

2.1 Generation

Generate Constraints from Decision Trees

1. Add graph-related features to the data (e.g., node degree)
2. Train a decision tree on the (pseudo-)labeled data
3. Extract logical rules from the decision tree paths

```
{
  "constrains": [
    {
      "conditions": [
        {"feature_index": 1263, "op": "<=", "threshold": 0.0},
        {"feature_index": 205, "op": "<=", "threshold": 0.0}
      ],
      "predicted_class": 1,
      "meta": {"depth": 2, "y_label_number": "98 v.s. 243"}
    },
    ...
  ],
  "additional_attributes": {
    "0:1433": "node_features",
    "1433": "node_degree",
    "1434": "clustering_coefficient"
  }
}
```

All of that from 0.2 percent of the dataset, so the constrains are built for every experiment.

Graph related features

For node level: node features, node degree, clustering coefficient

For graph level: mean node features, mean node degree

There have been used trees of depth 2 and 3.

2.2 Usage

I have come up with 2 main ways to translate logical rules into derivative features:
RuleBased (Fuzzy logic) and distance based (my own)

Fuzzy logic

Rule Evaluation: For each anomaly rule:

- Evaluate all conditions using soft logic
- Combine them with soft AND

Combine Rules: Apply soft OR over all rules

Output: A scalar violation score per node (higher values indicate stronger constraint violation for a chosen class of constrains)

So basically its about exchanging logical operations with derivative operations. those are:

Comparison operators:

- $x \leq t \rightarrow \sigma(t - x)$
- $x > t \rightarrow \sigma(x - t)$

Logic gates:

- $A \wedge B \rightarrow \prod_i A_i$
- $A \vee B \rightarrow 1 - \prod_i (1 - A_i)$

and in the end i multiply it by a lambda to regulate the importance of the constrains

Distance based

Thre idea was in imagining constrains as logical spaces and saying: there is a difference of a point being far away in "normalbone or squished in between normal spaces in the grey zone or whatever. so i have hased my method on distances.

Rules are divided into two groups, normality rules and anomaly rules (based on which class they predict)

For a sample $x \in \mathbb{R}^d$ and a constraint with conditions, the distance to the boundary is:

$$d(x, \text{constraint}) = \min_{c \in \text{conditions}} (x_{c.f} - c.\text{threshold})$$

where $c.f$ is the feature index and $c.\text{threshold}$ is the threshold from the condition.

For a group of rules (normality or anomaly):

$$\text{group-dist}(x, G) = \frac{\lambda}{|G|} \sum_{r \in G} d(x, r)$$

where λ is the scaling factor and $|G|$ is the number of rules in the group.

$$\text{satisfies}(x, r) = \begin{cases} \text{True} & \text{if } x \text{ satisfies ALL conditions in } r \\ \text{False} & \text{otherwise} \end{cases}$$

The main scoring function computes:

$$\text{group_diff} = \text{group_dist}(x, \text{normality}) - \text{group_dist}(x, \text{anomaly})$$

Then three cases are handled:

1. x in normal region (satisfies a normality rule):

$$\text{score} = -d_{\text{norm}} \cdot d_{\text{min,anom}} + \text{group_diff}$$

where d_{norm} is distance to normality boundary, $d_{\text{min,anom}}$ is min distance to anomaly rules.

2. x in anomaly region (satisfies an anomaly rule):

$$\text{score} = d_{\text{anom}} \cdot d_{\text{min,norm}} + \text{group_diff}$$

3. x in grey zone (satisfies neither):

$$\text{score} = \text{group_diff}$$

Finally, sigmoid is applied to get the final score. Higher score $\rightarrow$ more likely to be anomalous.

So by these rules the importance of a datapoint being in a normal/anormal region is handled quadratically while the importance of it being the the grey zone (perhaps close a near normal region) is linear

3 which loss?

For now the ones from the presentation:

Variant 1 – Additive:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n \|z_i - c\|^2 + \frac{1}{n} \sum_{i=1}^n L_{\text{cons}}(i)$$

Variant 2 – Weighting:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n (\|z_i - c\|^2 * (1 + L_{\text{cons}}(i)))$$

Variant 3 – Ignoring suspicious nodes:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n (\|z_i - c\|^2 * (1 - L_{\text{cons}}(i)))$$

Here, $L_{\text{cons}}(i)$ denotes the soft rule violation for node i .

For graph level its pretty much the same.

4 how were the experiments run?

I have chosen a bunch of parameters on which basis i have ran models without constrains and with to compare them. So for each model without constrains with its set of parameters (like number of layers, hidden dim usw) there are many logic models with the same parameters but also with their varying constrains parameters (tree depth, l factor, usw)

For node level:

The varying parameters across all models are:

1. "hidden_dim: [4, 8, 16, 32, 64, 128, 256, 512]
2. "num_layers: [2, 3, 4, 5]

For logic:

1. "lfactor: [1, 0.1, 0.5, 0.01, 0.001],
2. "decision_tree: ["max_depth: 3, "max_depth: 2,],
3. "model_train: ["logic_add_nl_ocgin", "logic_weight_nl_ocgin", "logic_ignore_sus_nl_ocgin"],
(the three losses)
4. "constrains_handler: [Fuzze logic, Distance Based]

default config is (params used across all models)

```
default_config = {  
    "lr": 1e-3,  
    "epochs": 50,  
    "batch_size": 32,  
    "save_logs": False,  
}
```

For graph level: The same when i think about it

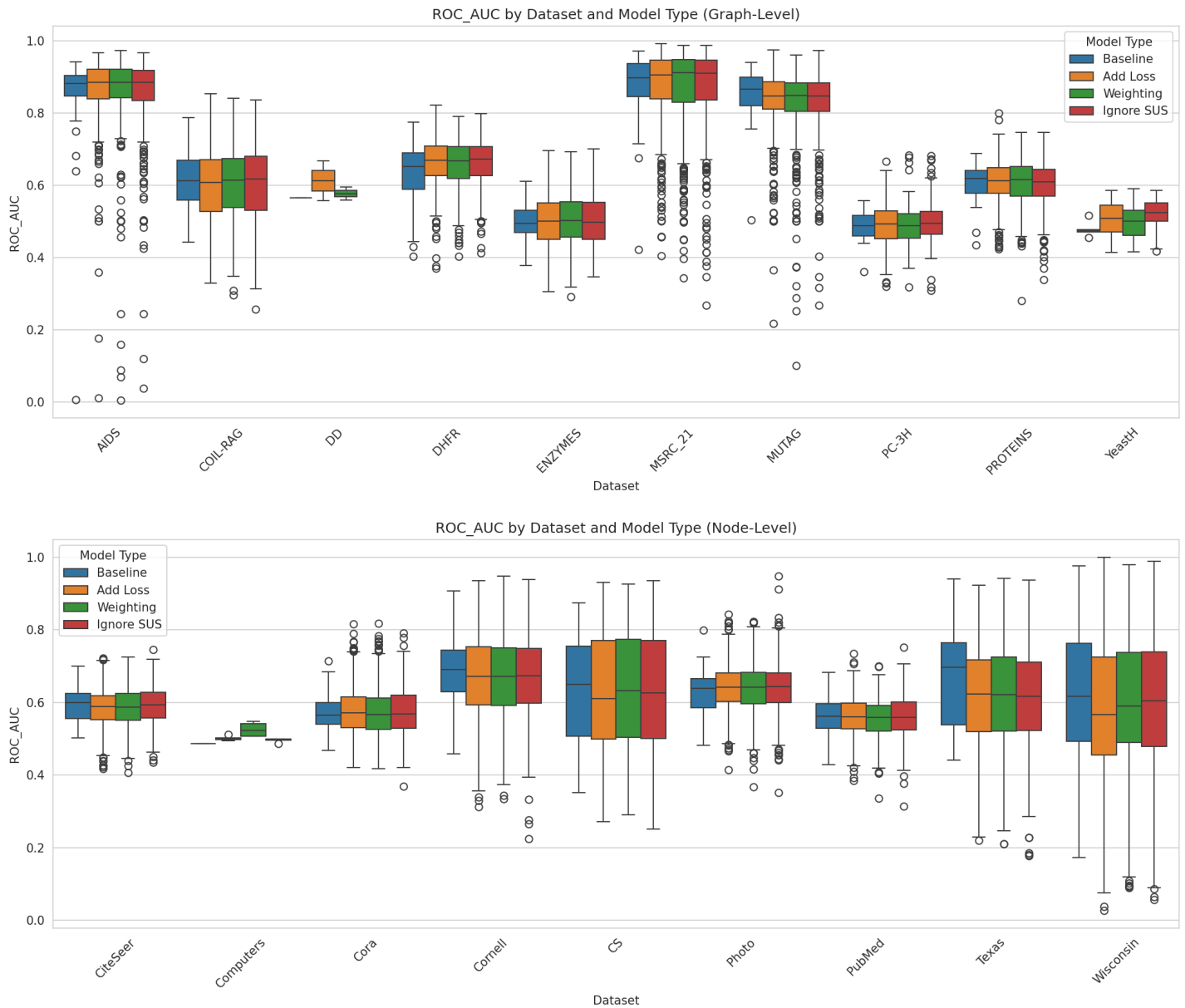
NOTICE

since some of the datasets were pretty big i have not ran for all models all parameters (ecpesially for graph level some datasets were stopped are reaching 32-64 hidden dim, since already that took like 12 hours for one dataset. if there is a need it will be done later)

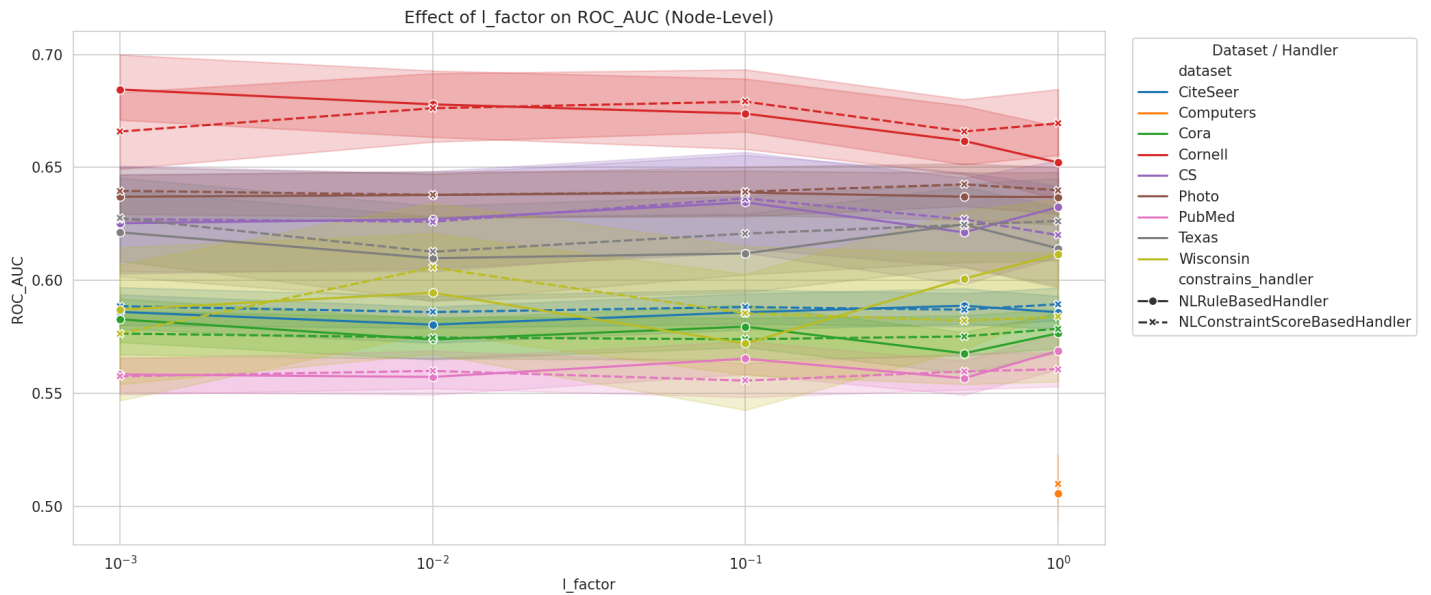
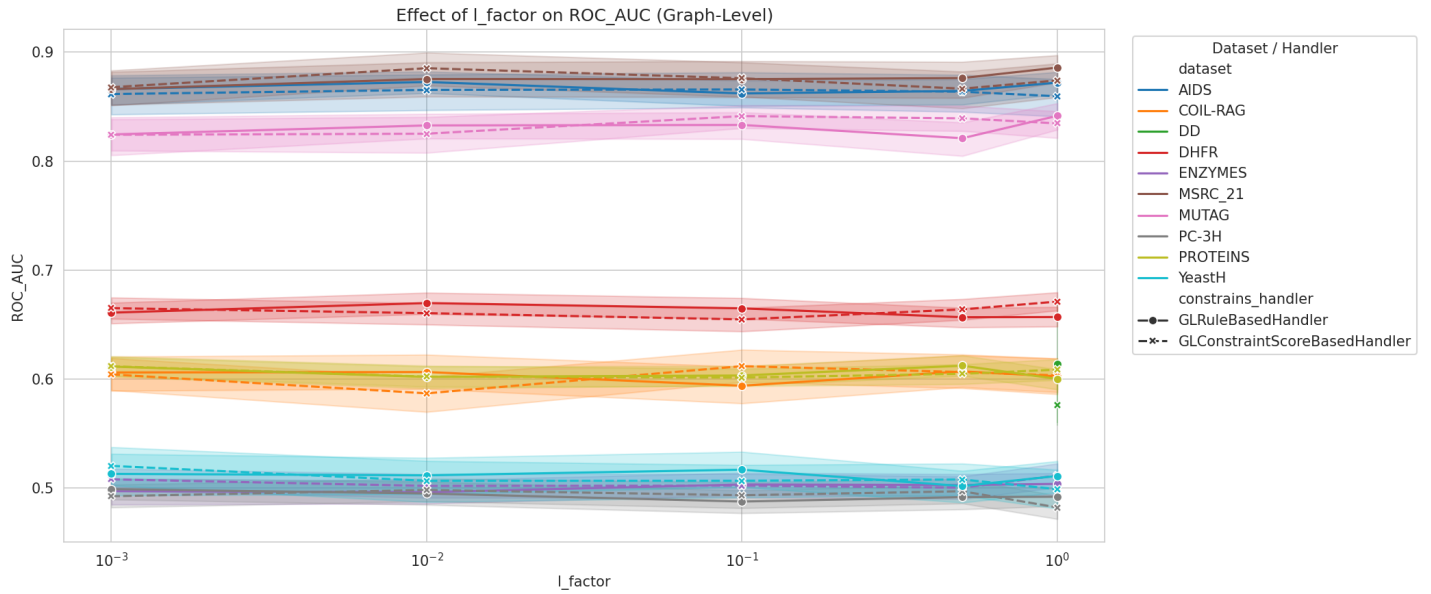
5 The results

The results come with the title "well, shit." the behavior of over 20 000 experiments look random. and looking and the model architecure the baseline outperforms me. (by ROC AUC)

This ones are about comparing models across datasets (sorry but youll have to zoom in)

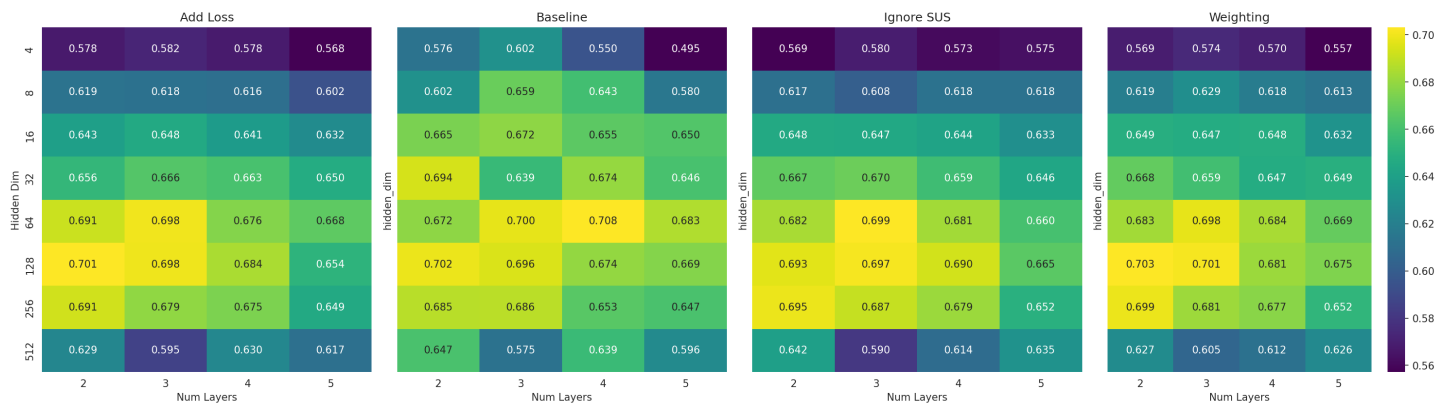


This one is about the l factor and how it influences performance (it really doesn't seem to)



And this one is about aggregated overall performance by architecture

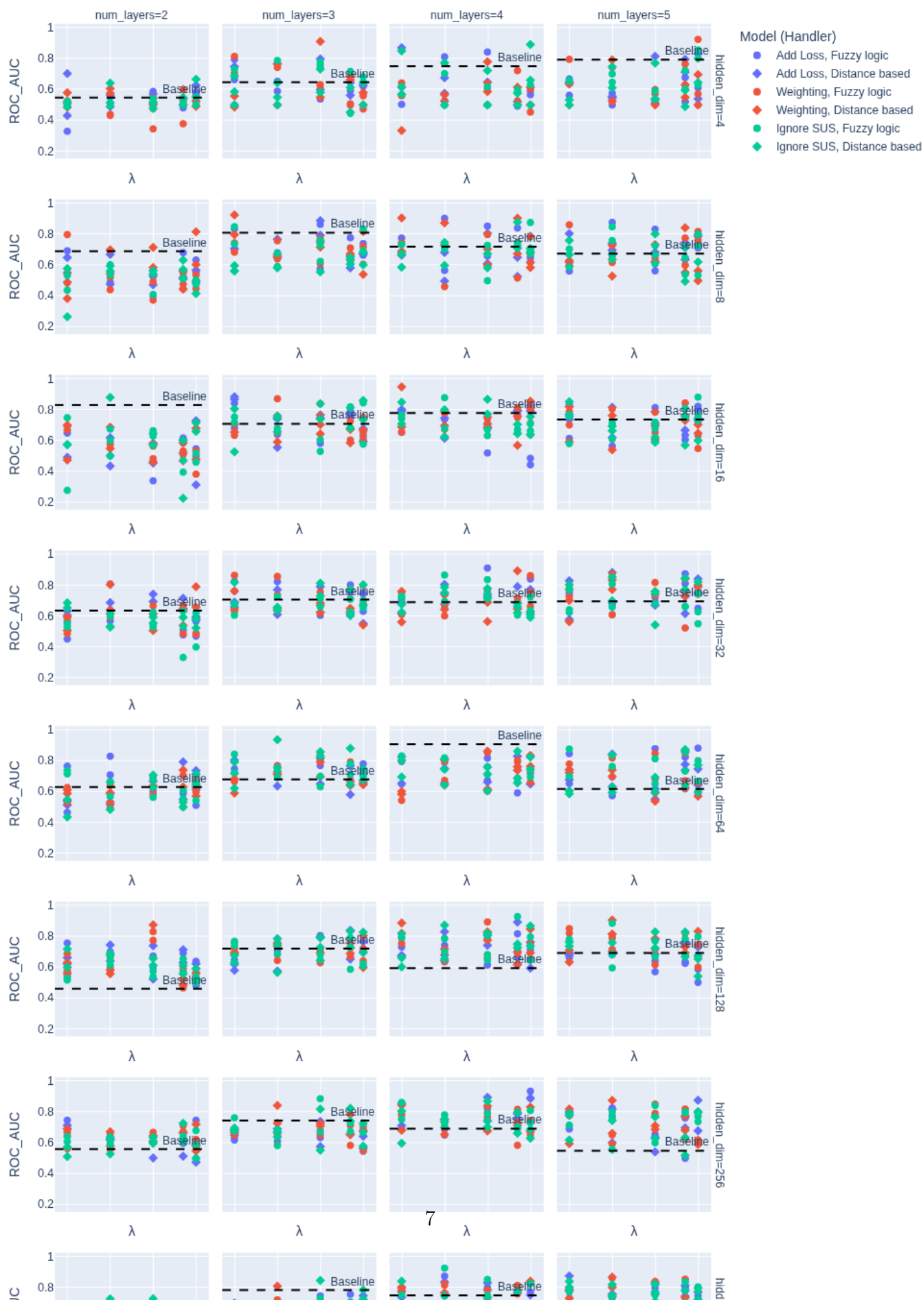
Mean ROC_AUC by Architecture (hidden_dim x num_layers)



I have also made a bucnh of detailed plots for every dataset but that like a lot of info

But here is for example, Cornell is typical and AIDS was interesting

Effect of λ on ROC_AUC - Cornell



Effect of λ on ROC_AUC - AIDS



Ah yeah, and i have not found a statistical difference between Handlers (Mann whitney U):

Handler comparison: Fuzzy median=0.6224, Distance median=0.6230, p-value=4.5592e-01

And between different models

6 My idea?

It seems like the constrains literally dont influence anything. it might be because of a error in implementation, so i will have to debug that.

And i have realized that i have used the constrains while the training process but i did not while forecasting. my next idea would be to do that, since it makes sence. other then that i guess, different approaches